

Algoritmos aleatorizados

IIC2283 – Diseño y Análisis de Algoritmos

Departamento de Ciencia de la Computación
Pontificia Universidad Católica de Chile

2025-2

Algoritmos aleatorizados

En el curso nos hemos centrado en el estudio de algoritmos determinísticos:

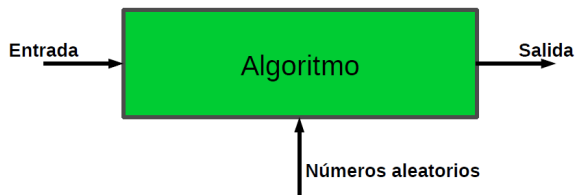
- ▶ Reciben una entrada y producen una salida correspondiente.
- ▶ Dada la misma entrada, el procedimiento para obtenerla (algoritmo) siempre realiza los mismos pasos.



- ▶ Nos centramos en demostrar:
 - ▶ que el algoritmo resuelve el problema correctamente (siempre) y
 - ▶ rápidamente (es decir, en un número acotado de pasos).

Algoritmos aleatorizados

- ▶ Un algoritmo aleatorio toma decisiones aleatorias durante su ejecución
- ▶ Además de una entrada, se tiene una fuente de números aleatorios, que permiten tomar las decisiones aleatorias



- ▶ Para una misma entrada, el proceso para obtener la salida puede ser distinto cada vez

Algoritmos aleatorizados

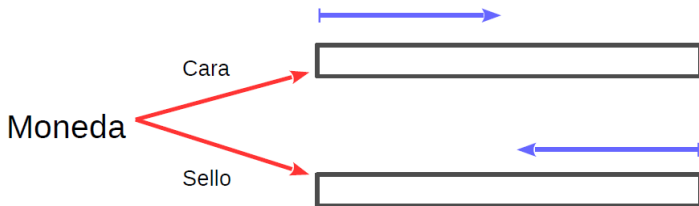
- ▶ Los algoritmos aleatorizados también se conocen como algoritmos aleatorios, algoritmos probabilísticos, y sus versiones en inglés randomized algorithms y probabilistic algorithms
- ▶ Al diseñar un algoritmo aleatorizado hacemos un análisis para mostrar que su comportamiento será bueno en el caso esperado (es decir, con alta probabilidad).
- ▶ La probabilidad aquí sólo afecta a la generación de números aleatorios, la idea es hacer a los algoritmos más independientes de las entradas
- ▶ Dado que el algoritmo toma decisiones aleatorias, es más difícil para un adversario generar un caso que produzca la mayor cantidad de trabajo al algoritmo

Algoritmos aleatorizados

Ejemplo 1: búsqueda lineal en un arreglo desordenado

Supongamos un algoritmo de búsqueda lineal en un arreglo desordenado $A[1..n]$, el cual lanza una moneda antes de comenzar:

- ▶ Si sale cara, se busca desde la posición 1 hasta la n .
- ▶ Si sale sello, se busca desde la posición n hasta la 1.



Algoritmos aleatorizados

Ejemplo 1: búsqueda lineal en un arreglo desordenado

- ▶ Note que para buscar el elemento que está en la posición i del arreglo ($1 \leq i \leq n$), la cantidad esperada de celdas del arreglo visitadas es:

$$\frac{i}{2} + \frac{n - i + 1}{2} = \frac{n + 1}{2}$$

- ▶ El costo de búsqueda esperado es $\frac{n+1}{2}$ celdas visitadas, sin necesidad de asumir nada respecto a la distribución de las búsquedas (tal como sí es necesario hacer con un análisis de caso promedio)
- ▶ Con ese simple cambio al algoritmo de búsqueda lineal, el caso que garantiza mayor cantidad de trabajo (en cantidad de celdas del arreglo visitadas) es buscar el elemento $A[n/2]$

Algoritmos aleatorizados

Existen dos tipos principales de algoritmos aleatorizados

- ▶ **Monte Carlo:** el algoritmo siempre entrega un resultado, pero hay una probabilidad de que sea incorrecto. Garantizan tiempo de ejecución de peor caso. Suelen ser acompañados por un análisis de error. La probabilidad de error puede reducirse ejecutando el algoritmo múltiples veces, de forma independiente
- ▶ **Las Vegas:** si el algoritmo entrega un resultado es correcto, pero hay una probabilidad de que no entregue resultado. Es decir, no hay garantías de tiempo de ejecución de peor caso. Suelen ser acompañados por un análisis de tiempo esperado.

¿Cuáles son las ventajas de los algoritmos aleatorizados?

Existen problemas para los cuales los algoritmos aleatorizados son más eficientes que los algoritmos usuales (sin una componente aleatoria)

- ▶ Por ejemplo, el problema de verificar si un número es primo

Existen problemas para los cuales los únicos algoritmos eficientes conocidos son aleatorizados

- ▶ Por ejemplo, el problema de verificar si dos polinomios en varias variables son equivalentes

Vamos a ver en detalle estos ejemplos . . .

Además, los algoritmos se vuelven menos dependientes de las entradas y sus distribuciones

Ejemplo 2: Encontrar un 1 en un arreglo de bits

Dado un arreglo de bits $B[1..n]$, el cual contiene $n/2$ 0's, y $n/2$ 1s, se quiere encontrar alguna posición i tal que $A[i] = 1$

Solución tipo Las Vegas

Find_1_LV($B[1..n]$)

repeat

Generar un valor i con distribución uniforme en $[1..n]$

until $B[i] = 1$

return i

- ▶ Este algoritmo tiene éxito con probabilidad 1
- ▶ Sin embargo, el tiempo de ejecución (cantidad de iteraciones que realiza) de peor caso no está acotado, pudiendo ser arbitrariamente alto

Ejemplo 2: Encontrar un 1 en un arreglo de bits

Análisis de la cantidad esperada de iteraciones que realiza el algoritmo

- ▶ El algoritmo tiene éxito en la primera iteración con prob. $1/2$, y 1 celda del arreglo visitada
- ▶ Tiene éxito en la segunda iteración con prob. $1/4$ y costo 2
- ▶ Tiene éxito en la tercera iteración con prob. $1/8$, y costo 3
- ▶ Tiene éxito en la i -ésima iteración con prob. $1/2^i$, y costo i
- ▶ Sea X la variable aleatoria que indica la cantidad de celdas de B visitadas
- ▶ El tiempo de ejecución esperado es la esperanza de X :

$$E(X) = \sum_{i=1}^{\infty} \frac{i}{2^i} = 2.$$

- ▶ Intuitivamente, si existen 50% de 1s, es probable que en dos accesos aleatorios encontremos uno de ellos

Ejemplo 2: Encontrar un 1 en un arreglo de bits

Solución tipo Monte Carlo

Find_1_MC($B[1..n]$, k)

$j \leftarrow 0$

repeat

 Generar un valor i con distribución uniforme en $[1..n]$

$j \leftarrow j + 1$

until $j = k$ **or** $B[i] = 1$

return i

- ▶ Este algoritmo recibe un parámetro k , que permite limitar el tiempo de ejecución a $O(k)$
- ▶ No siempre encuentra una posición que contiene un 1, dado que ejecuta a lo más k iteraciones

Ejemplo 2: Encontrar un 1 en un arreglo de bits

- ▶ El algoritmo puede no responder correctamente
- ▶ Para que un algoritmo de este tipo sea utilizable, debería ser acompañado de un análisis de probabilidad de error
- ▶ Si $k = 1$, el algoritmo tiene éxito con prob. $1/2 = 1 - 1/2$.
- ▶ Si $k = 2$, el algoritmo tiene éxito con prob. $3/4 = 1 - 1/4$.
 - ▶ Esto es porque puede encontrarlo en la primera iteración (con prob. $1/2$), o en la segunda (prob. $1/4$). Sumando esas probabilidades tenemos $3/4$.
- ▶ Si $k = 3$, el algoritmo tiene éxito con prob. $7/8 = 1 - 1/8$.
 - ▶ Esto es porque puede encontrarlo en la primera iteración (con prob. $1/2$), en la segunda (prob. $1/4$), o en la tercera (con prob. $1/8$). Sumando esas probabilidades tenemos $7/8$.
- ▶ En general, dado k , el algoritmo tiene éxito con probabilidad

$$\sum_{i=1}^k \frac{1}{2^i} = 1 - \left(\frac{1}{2}\right)^k.$$

Ejemplo 3: el algoritmo de Freivals

El problema de chequear la multiplicación de matrices

Dadas 3 matrices A , B , y C de $n \times n$ cada una, se quiere determinar si se cumple que $A \times B = C$

- ▶ Diferente al problema de computar la multiplicación, ya que aquí nos entregan el resultado como entrada
- ▶ Si usamos multiplicación de matrices para chequear el resultado:
 - ▶ Algoritmo clásico: tiempo $\Theta(n^3)$
 - ▶ Algoritmo de Strassen (1969): tiempo $O(n^{2,81})$
 - ▶ Algoritmo de Coppersmith-Winograd (1987): tiempo $O(n^{2,376})$
 - ▶ Algoritmo de Vassilevska (2015): tiempo $O(n^{2,373})$
 - ▶ No está claro si se puede hacer en tiempo $O(n^2)$
- ▶ El algoritmo aleatorizado de Freivals permite hacer el chequeo en tiempo $\Theta(n^2)$, aunque con probabilidad de fallar en la respuesta
- ▶ Aplicaciones: verificación de programas, donde se quiere verificar que el resultado C producido por la implementación de un algoritmo corresponde a la multiplicación $A \times B$ o no

Ejemplo 3: el algoritmo de Freivalds

El algoritmo de Freivalds es una estrategia de tipo Monte Carlo para chequear la multiplicación de matrices

Freivalds(A, B, C)

Generar un vector binario aleatorio $r = (r_1, r_2, \dots, r_n)$.

$P \leftarrow A \cdot (B \cdot r) - C \cdot r$

if $P = (0, 0, \dots, 0)$ **then**

return **Sí**

else

return **No**

- ▶ Se usa la asociatividad de la multiplicación de matrices para hacer primero $B \cdot r$ en tiempo $\Theta(n^2)$, luego multiplicar por A en tiempo $\Theta(n^2)$, luego multiplicar $C \times r$ en tiempo $\Theta(n^2)$
- ▶ Restar los vectores resultantes y verificar si $P = \vec{0}$ en tiempo $\Theta(n)$
- ▶ Tiempo total $\Theta(n^2)$

Ejemplo 3: el algoritmo de Freivalds

- ▶ Freivalds se basa en el hecho de que *si se cumple $A \times B = C$, entonces también se cumple $A \times B \cdot r = C \cdot r$, para cualquier vector $r = (r_1, r_2, \dots, r_n)$*
- ▶ Lamentablemente, si $A \times B \neq C$, no necesariamente se cumple que $A \times B \cdot r \neq C \cdot r$
- ▶ Por ejemplo, se cumple que

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix} \neq \begin{bmatrix} 3 & 1 \\ 1 & 1 \end{bmatrix}$$

- ▶ Sin embargo también se cumple

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

- ▶ De esta manera, puede ocurrir que $P = (0, 0, \dots, 0)$ aún cuando $A \times B \neq C$. En ese caso, el algoritmo cometerá un error, respondiendo “**Sí**” en lugar de “**No**”

Ejemplo 3: el algoritmo de Freivalds

A continuación realizamos un análisis más detallado de errores que comete el algoritmo al responder:

- ▶ Si se cumple $A \times B = C$, entonces $A \times (B \cdot r) = C \cdot r$ siempre se va a cumplir —tal como lo mencionamos anteriormente— por lo que $A \times (B \cdot r) - C \cdot r = (0, 0, \dots, 0)$. Eso significa que el algoritmo siempre responde correctamente cuando la igualdad se cumple
- ▶ Si, por otro lado, se cumple $A \times B \neq C$ entonces tenemos dos casos:
 1. El algoritmo responde “**No**”: el algoritmo siempre es correcto en este caso, ya que significa que $P \neq (0, 0, \dots, 0)$
 2. El algoritmo responde “**Sí**”: como vimos anteriormente, se puede dar que $A \times (B \cdot r) = C \cdot r$ aún cuando $A \times B \neq C$. El algoritmo comete un error en este caso, ya que debería haber respondido “**No**”

Ejemplo 3: el algoritmo de Freivalds

En resumen:

- ▶ El algoritmo siempre es correcto cuando responde “**No**”
- ▶ Puede equivocarse cuando responde “**Sí**”
- ▶ Acotamos a continuación la probabilidad de una respuesta incorrecta.
- ▶ Es decir, si el algoritmo responde “**Sí**”, ¿**Cuál es la probabilidad de que esa respuesta sea errónea?**
- ▶ Supongamos que $A \times B \neq C$ y el algoritmo erróneamente responde “**Sí**”
- ▶ Definimos

$$P = A \cdot (B \cdot r) - C \cdot r = (A \times B - C) \cdot r = D \times r = (p_1, p_2, \dots, p_n),$$

en donde $D = A \times B - C$ y $p_k = 0$ para $k = 1, \dots, n$

Ejemplo 3: el algoritmo de Freivalds

- ▶ Ya que $A \times B \neq C$, entonces al menos un elemento $d_{i,j}$ de la matriz D es distinto de cero
- ▶ Por definición de multiplicación de matrices y vectores, tenemos que

$$p_i = \sum_{k=1}^n d_{i,k} \cdot r_k = d_{i,1} \cdot r_1 + \cdots + d_{i,j} \cdot r_j + \cdots + d_{i,n} \cdot r_n = d_{i,j} \cdot r_j + y.$$

- ▶ Determinamos a continuación $\Pr \{p_i = 0\}$ dado que $d_{i,j} \neq 0$.
- ▶ Usando el Teorema de Bayes, podemos decir que:

$$\begin{aligned} \Pr \{p_i = 0\} &= \Pr \{p_i = 0 | y = 0\} \cdot \Pr \{y = 0\} \\ &\quad + \Pr \{p_i = 0 | y \neq 0\} \cdot \Pr \{y \neq 0\}, \end{aligned}$$

en donde $\Pr \{p_i = 0 | y = 0\} = \Pr \{r_j = 0\}$ ya que $d_{i,j} \neq 0$.

Ejemplo 3: el algoritmo de Freivalds

- ▶ Dado que los elementos r_i del vector r se elijen de forma aleatoria con probabilidad $1/2$ de ser 0 o 1, tenemos que

$$\Pr \{p_i = 0 | y = 0\} = 1/2$$

- ▶ Por otro lado, tenemos que

$$\Pr \{p_i = 0 | y \neq 0\} = \Pr \{r_j = 1 \wedge d_{i,j} = -y\},$$

lo cual claramente es $\leq \Pr \{r_j = 1\} = 1/2$.

- ▶ Reemplazando esto en la ecuación de arriba produce

$$\begin{aligned}\Pr \{p_i = 0\} &\leq 1/2 \cdot \Pr \{y = 0\} + 1/2 \cdot \Pr \{y \neq 0\} \\ &= 1/2 \cdot \Pr \{y = 0\} + 1/2(1 - \Pr \{y = 0\}) \\ &= 1/2.\end{aligned}$$

Ejemplo 3: el algoritmo de Freivalds

- ▶ Eso es para cada p_i . Si consideramos el vector P completo, tenemos que

$$\begin{aligned}\Pr \left\{ P = \vec{0} \right\} &= \Pr \{ p_1 = 0 \wedge p_2 = 0 \wedge \cdots \wedge p_n = 0 \} \\ &\leq \Pr \{ p_i = 0 \} \\ &= 1/2.\end{aligned}$$

- ▶ Así, el algoritmo de Freivalds responde incorrectamente con a lo más esa probabilidad

Ejemplo 3: el algoritmo de Freivalds

- ▶ Se puede reducir esta probabilidad de error significativamente repitiendo el proceso de Freivalds $k \geq 1$ veces,
- ▶ Cada vez usando vectores r independientes de los anteriores
- ▶ De esta manera, cada ejecución es independiente de la anterior
- ▶ Si el algoritmo responde “**Si**” las k veces, la probabilidad de error es $\leq 1/2^k$ —la independencia de las ejecuciones es clave
- ▶ Si en alguno de los pasos responde “**No**”, el proceso se detiene con esa respuesta —recuerde que el algoritmo no falla en ese caso
- ▶ El tiempo de ejecución es $\Theta(kn^2)$.
 - ▶ Para $k = \lg n$, el tiempo es $\Theta(n^2 \lg n)$ —menor asintóticamente que el mejor algoritmo conocido de multiplicación de matrices— con probabilidad de error $\leq 1/2^{\lg n} = 1/n$.
 - ▶ Para $k = \Theta(\lg^q n)$, con $q \in \Theta(1)$, el tiempo todavía es asintóticamente menor al mejor algoritmo conocido para multiplicar matrices, y la probabilidad de error es $\leq 1/n^q$

Ejemplo 4: equivalencia de polinomios

Consideramos polinomios en $\mathbb{Q}$

Suponemos inicialmente que un polinomio es una expresión de la forma:

$$p(x) = \sum_{i=1}^k \prod_{j=1}^{\ell_i} (a_{i,j}x + b_{i,j})$$

donde cada $a_{i,j}, b_{i,j} \in \mathbb{Q}$

La forma canónica de $p(x)$ es una expresión de la forma:

$$p(x) = \sum_{i=0}^{\ell} c_i x^i$$

donde cada $c_i \in \mathbb{Q}$ y $\ell \leq \max\{\ell_1, \dots, \ell_k\}$

Si $c_{\ell} \neq 0$, entonces $p(x)$ no es el polinomio nulo y su grado es ℓ

Ejemplo 4: equivalencia de polinomios

Dados dos polinomios $p(x)$ y $q(x)$, queremos verificar si son idénticos.

- ▶ Para cada $a \in \mathbb{Q}$, se tiene que $p(a) = q(a)$

¿Cómo podemos resolver este problema?

- ▶ La operación básica a contar es la suma y multiplicación de números racionales

Un algoritmo para la equivalencia de polinomios

EquivPol($p(x)$, $q(x)$)

transforme $p(x)$ es su forma canónica $\sum_{i=0}^k c_i x^i$

transforme $q(x)$ es su forma canónica $\sum_{i=0}^{\ell} d_i x^i$

if $k \neq \ell$ **then return** no

else

for $i := 0$ **to** k **do**

if $c_i \neq d_i$ **then return** no

return sí

Un algoritmo para la equivalencia de polinomios

Ejercicio

Muestre que el algoritmo anterior en el peor caso es $O(n^2)$, donde $n = |p(x)| + |q(x)|$

¿Es posible resolver este problema utilizando un menor número de operaciones?

Un algoritmo aleatorizado para la equivalencia de polinomios

Suponga que:

$$p(x) = \sum_{i=1}^k \prod_{j=1}^{r_i} (a_{i,j}x + b_{i,j})$$

$$q(x) = \sum_{i=1}^{\ell} \prod_{j=1}^{s_i} (c_{i,j}x + d_{i,j})$$

Utilizamos el siguiente algoritmo **aleatorizado**:

EquivPolAleatorizado($p(x)$, $q(x)$)

$K := 1 + \text{máx}\{r_1, \dots, r_k, s_1, \dots, s_{\ell}\}$

escoja al azar y con distribución uniforme un elemento a
del conjunto de números naturales $\{1, \dots, 100 \cdot K\}$

if $p(a) = q(a)$ then return sí

else return no

Un algoritmo aleatorizado para la equivalencia de polinomios

El algoritmo sólo necesita realizar $O(n)$ operaciones, donde $n = |p(x)| + |q(x)|$

- ▶ Ya que necesita calcular $p(a)$ y $q(a)$

Pero el algoritmo puede dar una respuesta equivocada

- ▶ Eso significa que es tipo Monte Carlo
- ▶ ¿Cuál es la probabilidad de error?

Calculando la probabilidad de error

Sean $p(x)$ y $q(x)$ dos polinomios dados como entrada a

EquivPolAleatorizado

- ▶ Si los polinomios $p(x)$ y $q(x)$ son equivalentes, entonces el algoritmo responde **sí** sin cometer error
- ▶ Si los polinomios $p(x)$ y $q(x)$ no son equivalentes, el algoritmo puede responder **sí** al sacar al azar un elemento $a \in \{1, \dots, 100 \cdot K\}$ tal que $p(a) = q(a)$

Esto significa que a es una raíz del polinomio $r(x) = p(x) - q(x)$

Calculando la probabilidad de error

$r(x)$ no es el polinomio nulo y es de grado a lo más K

► Por lo tanto $r(x)$ tiene a lo más K raíces en $\mathbb{Q}$

Concluimos que:

$$\begin{aligned}\Pr(a \text{ sea una raíz de } r(x)) &\leq \frac{K}{100 \cdot K} \\ &= \frac{1}{100}\end{aligned}$$

Un mejor algoritmo aleatorizado

La probabilidad de error del algoritmo está acotada por $\frac{1}{100}$

- ▶ ¿Es aceptable esta probabilidad?

Ejercicio

De un algoritmo que resuelva el problema de equivalencia de polinomios, que en el peor caso sea $O(n)$ y que tenga una probabilidad de error acotada por $\frac{1}{100^{10}}$

¿Confiaría en este algoritmo lineal?

- ▶ ¿Para qué probabilidad estaría dispuesto a confiar?

Una solución para el ejercicio

Suponga que $p(x)$ y $q(x)$ son de la forma definida en la versión anterior de **EquivPolAleatorizado**.

EquivPolAleatorizado($p(x)$, $q(x)$)

$K := 1 + \text{máx}\{r_1, \dots, r_k, s_1, \dots, s_\ell\}$

$A := \{1, \dots, 100 \cdot K\}$

$total := 0$

for $i := 1$ **to** 10 **do**

 escoja al azar y con distribución uniforme un elemento a en A

if $p(a) = q(a)$ **then** $total = total + 1$

if $total = 10$ **then return** sí

return no

Ejemplo 5: una definición general de polinomios

Consideramos polinomios en varias variables en $\mathbb{Q}$

Un monomio es una expresión de la forma $cx_1^{\ell_1} \cdots x_n^{\ell_n}$, donde $c \in \mathbb{Q}$ y cada $\ell_i \in \mathbb{N}$

Un monomio $cx_1^{\ell_1} \cdots x_n^{\ell_n}$ es nulo si $c = 0$

▶ No es nulo si $c \neq 0$

El grado de un monomio $cx_1^{\ell_1} \cdots x_n^{\ell_n}$ no nulo es $\ell_1 + \cdots + \ell_n$

Ejemplo 5: una definición general de polinomios

Un polinomio es una expresión de la forma:

$$p(x_1, \dots, x_n) = \sum_{i=1}^{\ell} \prod_{j=1}^{m_i} \left(\sum_{k=1}^n a_{i,j,k} x_k + b_{i,j} \right)$$

donde cada $a_{i,j,k} \in \mathbb{Q}$ y cada $b_{i,j} \in \mathbb{Q}$

Ejemplo 5: una definición general de polinomios

La forma canónica de un polinomio $p(x_1, \dots, x_n)$ es única, y es igual a 0 o a una suma de monomios que satisface las siguientes propiedades:

- ▶ cada monomio en la forma canónica es de la forma $cx_1^{\ell_1} \cdots x_n^{\ell_n}$ con $c \neq 0$
- ▶ si $cx_1^{\ell_1} \cdots x_n^{\ell_n}$ y $dx_1^{m_1} \cdots x_n^{m_n}$ son dos monomios distintos en la forma canónica, entonces $\ell_i \neq m_i$ para algún $i \in \{1, \dots, n\}$

Un polinomio $p(x_1, \dots, x_n)$ es nulo si su forma canónica es 0

El grado de un polinomio $p(x_1, \dots, x_n)$ no nulo es el mayor grado de los monomios en su forma canónica.

Equivalencia de polinomios en varias variables

Dos polinomios $p(x_1, \dots, x_n)$ y $q(x_1, \dots, x_n)$ son idénticos si para cada secuencia $a_1, \dots, a_n \in \mathbb{Q}$ se tiene que:

$$p(a_1, \dots, a_n) = q(a_1, \dots, a_n)$$

Nuevamente queremos verificar si dos polinomios son idénticos.

Equivalencia de polinomios en varias variables

¿Podemos verificar en tiempo polinomial si dos polinomios en varias variables son equivalentes?

Tenemos un problema: calcular la forma canónica de un polinomio toma tiempo exponencial

Pero existe un algoritmo aleatorizado eficiente para este problema.

- ▶ Esto no es trivial ya que un polinomio $p(x_1, \dots, x_n)$ puede tener una cantidad infinita de raíces
 - ▶ Por ejemplo: $p(x_1, x_2) = (x_1 - 1)(x_2 - 3)$
- ▶ El ingrediente esencial es el lema de Schwartz-Zippel

El ingrediente principal

Lema de Schwartz-Zippel

Sea $p(x_1, \dots, x_n)$ un polinomio no nulo de grado k , y sea A un subconjunto finito y no vacío de $\mathbb{Q}$. Si $a_1, \dots, a_n$ son elegidos de manera uniforme e independiente desde A , entonces

$$\Pr(p(a_1, \dots, a_n) = 0) \leq \frac{k}{|A|}$$

Un algoritmo aleatorizado para la equivalencia de polinomios en varias variables

Vamos a dar un algoritmo aleatorizado eficiente para el problema de verificar si dos polinomios en varias variables son equivalentes

Suponga que la entrada del algoritmo está dada por los siguientes polinomios:

$$p(x_1, \dots, x_n) = \sum_{i=1}^{\ell} \prod_{j=1}^{r_i} \left(\sum_{k=1}^n a_{i,j,k} x_k + b_{i,j} \right)$$
$$q(x_1, \dots, x_n) = \sum_{i=1}^m \prod_{j=1}^{s_i} \left(\sum_{k=1}^n c_{i,j,k} x_k + d_{i,j} \right)$$

Un algoritmo aleatorizado para la equivalencia de polinomios en varias variables

EquivPolAleatorizado($p(x_1, \dots, x_n)$, $q(x_1, \dots, x_n)$)

$K := 1 + \text{máx} \{r_1, \dots, r_\ell, s_1, \dots, s_m\}$

$A := \{1, \dots, 100 \cdot K\}$

sea $a_1, \dots, a_n$ una secuencia de números elegidos de
manera uniforme e independiente desde A

if $p(a_1, \dots, a_n) = q(a_1, \dots, a_n)$ **then return** sí

else return no

Utilizando el lema de Schwartz-Zippel

Vamos a calcular la probabilidad de error del algoritmo:

- ▶ Si los polinomios $p(x_1, \dots, x_n)$ y $q(x_1, \dots, x_n)$ son equivalentes, entonces el algoritmo responde **sí** sin cometer error
- ▶ Si los polinomios $p(x_1, \dots, x_n)$ y $q(x_1, \dots, x_n)$ no son equivalentes, el algoritmo puede responder **sí** al escoger una secuencia de números $a_1, \dots, a_n$ desde A tales que $p(a_1, \dots, a_n) = q(a_1, \dots, a_n)$
 - ▶ Donde $A = \{1, \dots, 100 \cdot K\}$

Esto significa que $(a_1, \dots, a_n)$ es una raíz del polinomio $r(x_1, \dots, x_n) = p(x_1, \dots, x_n) - q(x_1, \dots, x_n)$

Utilizando el lema de Schwartz-Zippel

$r(x_1, \dots, x_n)$ no es el polinomio nulo y es de grado t con $t < K$

► Dado que $K = 1 + \max\{r_1, \dots, r_\ell, s_1, \dots, s_m\}$

Utilizando el lema de Schwartz-Zippel obtenemos:

$$\Pr(r(a_1, \dots, a_n) = 0) \leq \frac{t}{|A|} < \frac{K}{|A|} = \frac{K}{100 \cdot K} = \frac{1}{100}$$

La probabilidad de error del algoritmo está entonces acotada por $\frac{1}{100}$

Un mejor algoritmo aleatorizado para el problema general

Ejercicio

De un algoritmo aleatorizado que resuelva el problema de equivalencia de polinomios en varias variables.

- ▶ La probabilidad de error del algoritmo debe estar acotada por $\frac{1}{100^{10}}$
- ▶ Debe existir una constante c tal que el algoritmo en el peor caso es $O(m^c)$, donde m es el tamaño de la entrada
 - ▶ Si consideramos $p(x_1, \dots, x_n)$ y $q(x_1, \dots, x_n)$ como palabras sobre un cierto alfabeto, entonces $m = |p(x_1, \dots, x_n)| + |q(x_1, \dots, x_n)|$
 - ▶ Recuerdo que la operación básica a contar es la suma y multiplicación de números racionales.

Ejemplo 6: Calculando la mediana de una lista de números enteros

Suponga que n es un número impar y $L[1 \dots n]$ es una lista de números enteros sin elementos repetidos.

- ▶ Recuerde que usamos la notación $L[1 \dots n]$ para indicar que L es una lista con n elementos

$L[i]$ es la mediana de L si:

$$\begin{aligned} |\{j \in \{1, \dots, n\} \mid L[j] < L[i]\}| &= \left\lfloor \frac{n}{2} \right\rfloor \\ |\{k \in \{1, \dots, n\} \mid L[k] > L[i]\}| &= \left\lfloor \frac{n}{2} \right\rfloor \end{aligned}$$

Ejemplo 6: Calculando la mediana de una lista de números enteros

Ejercicio

Construya un algoritmo que calcule la mediana de una lista $L[1 \dots n]$ y que en el peor caso sea $O(n \cdot \log_2(n))$

- ▶ Considere como la operación básica a contar la comparación de números enteros

Vamos a construir un algoritmo aleatorizado de tipo Las Vegas para este problema.

- ▶ ¡Este algoritmo funciona en tiempo lineal!

Un algoritmo aleatorizado para el cálculo de la mediana

Suponga que el procedimiento **Mergesort**(L) ordena una lista L utilizando el algoritmo Mergesort

- ▶ Recuerde que las listas son pasados por referencia en este curso

El siguiente procedimiento calcula la mediana de una lista de enteros $L[1 \dots n]$ (suponiendo que n es impar y L no tiene elementos repetidos):

CalcularMediana($L[1 \dots n]$)

if $n < 2001$ **then**

Mergesort(L)

return $L[\lceil \frac{n}{2} \rceil]$

else

 sea R una lista de $\lceil n^{\frac{3}{4}} \rceil$ números enteros escogido con
 distribución uniforme y de manera independiente desde L

Mergesort(R)

Un algoritmo aleatorizado para el cálculo de la mediana

$$d := R \left[\left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor \right]$$

$$u := R \left[\left\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \right\rceil \right]$$

$$S := \emptyset$$

$$m_d := 0$$

$$m_u := 0$$

for $i := 1$ **to** n **do**

if $d \leq L[i]$ **and** $L[i] \leq u$ **then** **Append**($S, [L[i]]$)

else if $L[i] < d$ **then** $m_d := m_d + 1$

else $m_u := m_u + 1$

if $m_d \geq \lceil \frac{n}{2} \rceil$ **or** $m_u \geq \lceil \frac{n}{2} \rceil$ **or**

$\text{Length}(S) > 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor$ **then return** *sin_resultado*

else

Mergesort(S)

return $S[\lceil \frac{n}{2} \rceil - m_d]$

El algoritmo es correcto y eficiente

Ejercicios

Demuestre lo siguiente:

1. Si **CalcularMediana**(L) retorna un número entero m , entonces m es la mediana de L
2. Si se tiene un procedimiento **LanzarMoneda**() que retorna 0 ó 1 con probabilidad $\frac{1}{2}$, entonces existe un algoritmo para construir R que invoca a este procedimiento a los más $c \cdot n^{\frac{3}{4}} \cdot \log_2(n)$ veces, donde c es una constante fija y n es el largo de la lista de entrada
 - ▶ Podemos suponer que **LanzarMoneda**() en el peor caso es $O(1)$
3. **CalcularMediana**(L) en el peor caso es $O(n)$, suponiendo que n es el largo de L y considerando todas las operaciones realizadas

¿Cuál es la probabilidad de no retornar un resultado?

La llamada **CalcularMediana**(L) puede no retornar un resultado

- ▶ El procedimiento en este caso retorna *sin_resultado*

Para que **CalcularMediana** pueda ser utilizado en la práctica la probabilidad que no entregue un resultado debe ser baja

- ▶ Vamos a demostrar esto

La probabilidad de no retornar resultado

Sea $L[1 \dots n]$ una lista de números enteros tal que $n \geq 2001$, n es impar y la mediana de L es m

Defina las siguientes variables aleatorias:

$$Y_1 = |\{i \in \{1, \dots, \lceil n^{\frac{3}{4}} \rceil\} \mid R[i] \leq m\}|$$

$$Y_2 = |\{i \in \{1, \dots, \lceil n^{\frac{3}{4}} \rceil\} \mid R[i] \geq m\}|$$

Estas son variables aleatorias dado que R es construido escogiendo elementos de L con distribución uniforme (y de manera independiente)

La probabilidad de no retornar resultado

Lema

CalcularMediana(L) *retorna sin_resultado si y sólo si alguna de las siguientes condiciones se cumple:*

1. $Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor$
2. $Y_2 \leq \lceil n^{\frac{3}{4}} \rceil - \left\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \right\rceil$
3. **Length**(S) $> 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor$

Ejercicio

Demuestre el lema.

La probabilidad de no retornar resultado

Tenemos entonces que la probabilidad de que **CalcularMediana(L)** retorne *sin_resultado* es igual a:

$$\Pr \left(Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor \vee \right. \\ \left. Y_2 \leq \left\lceil n^{\frac{3}{4}} \right\rceil - \left\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \right\rceil \vee \text{Length}(S) > 4 \cdot \left\lfloor n^{\frac{3}{4}} \right\rfloor \right)$$

Necesitamos entonces acotar superiormente $\Pr \left(Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor \right)$,
 $\Pr \left(Y_2 \leq \left\lceil n^{\frac{3}{4}} \right\rceil - \left\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \right\rceil \right)$ y $\Pr \left(\text{Length}(S) > 4 \cdot \left\lfloor n^{\frac{3}{4}} \right\rfloor \right)$

► Vamos a ver dos desigualdades muy útiles para acotar probabilidades

La desigualdad de Markov

Teorema

Sea X una variable aleatoria no negativa. Para cada $a \in \mathbb{R}^+$ se tiene que:

$$\Pr(X \geq a) \leq \frac{E(X)}{a}$$

Una demostración de la desigualdad de Markov

Suponemos que el recorrido de X es un conjunto finito $\Omega \subseteq \mathbb{R}_0^+$:

$$\begin{aligned} E(X) &= \sum_{r \in \Omega} r \cdot \Pr(X = r) \\ &= \left(\sum_{r \in \Omega : r < a} r \cdot \Pr(X = r) \right) + \left(\sum_{s \in \Omega : s \geq a} s \cdot \Pr(X = s) \right) \\ &\geq \sum_{s \in \Omega : s \geq a} s \cdot \Pr(X = s) \\ &\geq \sum_{s \in \Omega : s \geq a} a \cdot \Pr(X = s) \\ &= a \cdot \left(\sum_{s \in \Omega : s \geq a} \Pr(X = s) \right) \\ &= a \cdot \Pr(X \geq a) \end{aligned}$$

Concluimos que $\Pr(X \geq a) \leq \frac{E(X)}{a}$



La desigualdad de Chebyshev

Teorema

$$\Pr(|X - E(X)| \geq a) \leq \frac{\text{Var}(X)}{a^2}$$

Demostración. Utilizando la desigualdad de Markov obtenemos:

$$\begin{aligned}\Pr(|X - E(X)| \geq a) &= \Pr((X - E(X))^2 \geq a^2) \\ &\leq \frac{E((X - E(X))^2)}{a^2} \\ &= \frac{\text{Var}(X)}{a^2}\end{aligned}$$



Utilizando la desigualdad de Chebyshev

Lema

$$\Pr\left(Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor\right) \leq n^{-\frac{1}{4}}$$

Demostración: para cada $i \in \{1, \dots, \lceil n^{\frac{3}{4}} \rceil\}$, definimos una variable aleatoria X_i de la siguiente forma:

$$X_i = \begin{cases} 1 & R[i] \leq m \\ 0 & R[i] > m \end{cases}$$

Tenemos que:

$$Y_1 = \sum_{i=1}^{\lceil n^{3/4} \rceil} X_i$$

Utilizando la desigualdad de Chebyshev

Dado que la lista L no contiene elementos repetidos tenemos que:

$$\Pr(X_i = 1) = \frac{\lceil \frac{n}{2} \rceil}{n} = \frac{\frac{n-1}{2} + 1}{n} = \frac{1}{2} + \frac{1}{2 \cdot n}$$

De esto se deduce que:

$$\begin{aligned} E(X_i) &= \frac{1}{2} + \frac{1}{2 \cdot n} \\ \text{Var}(X_i) &= \left(\frac{1}{2} + \frac{1}{2 \cdot n} \right) \cdot \left(1 - \frac{1}{2} - \frac{1}{2 \cdot n} \right) \\ &= \left(\frac{1}{2} + \frac{1}{2 \cdot n} \right) \cdot \left(\frac{1}{2} - \frac{1}{2 \cdot n} \right) \\ &= \frac{1}{4} - \frac{1}{4 \cdot n^2} \end{aligned}$$

Utilizando la desigualdad de Chebyshev

Por lo tanto tenemos que:

$$\begin{aligned} E(Y_1) &= E\left(\sum_{i=1}^{\lceil n^{3/4} \rceil} X_i\right) \\ &= \sum_{i=1}^{\lceil n^{3/4} \rceil} E(X_i) \\ &= \sum_{i=1}^{\lceil n^{3/4} \rceil} \left(\frac{1}{2} + \frac{1}{2 \cdot n}\right) \\ &= \lceil n^{3/4} \rceil \cdot \left(\frac{1}{2} + \frac{1}{2 \cdot n}\right) \end{aligned}$$

Utilizando la desigualdad de Chebyshev

Para $i, j \in \{1, \dots, \lceil n^{3/4} \rceil\}$ tal que $i \neq j$ se tiene que X_i es independiente de X_j

Concluimos entonces que:

$$\begin{aligned}\text{Var}(Y_1) &= \text{Var}\left(\sum_{i=1}^{\lceil n^{3/4} \rceil} X_i\right) \\&= \sum_{i=1}^{\lceil n^{3/4} \rceil} \text{Var}(X_i) \\&= \sum_{i=1}^{\lceil n^{3/4} \rceil} \left(\frac{1}{4} - \frac{1}{4 \cdot n^2}\right) \\&= \lceil n^{3/4} \rceil \cdot \left(\frac{1}{4} - \frac{1}{4 \cdot n^2}\right) \\&\leq \frac{1}{4} \cdot \lceil n^{3/4} \rceil\end{aligned}$$

Utilizando la desigualdad de Chebyshev

Tenemos que:

$$\begin{aligned}\Pr\left(Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor\right) &\leq \Pr\left(Y_1 < \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}}\right) \\ &\leq \Pr\left(Y_1 < \frac{1}{2} \cdot \lceil n^{\frac{3}{4}} \rceil - n^{\frac{1}{2}}\right) \\ &\leq \Pr\left(Y_1 < \lceil n^{\frac{3}{4}} \rceil \cdot \left(\frac{1}{2} + \frac{1}{2 \cdot n}\right) - n^{\frac{1}{2}}\right) \\ &= \Pr\left(Y_1 < E(Y_1) - n^{\frac{1}{2}}\right) \\ &= \Pr\left(n^{\frac{1}{2}} < E(Y_1) - Y_1\right) \\ &\leq \Pr\left(|Y_1 - E(Y_1)| > n^{\frac{1}{2}}\right) \\ &\leq \Pr\left(|Y_1 - E(Y_1)| \geq n^{\frac{1}{2}}\right)\end{aligned}$$

Utilizando la desigualdad de Chebyshev

Por lo tanto, utilizando la desigualdad de Chebyshev concluimos que:

$$\begin{aligned}\Pr\left(Y_1 < \left\lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \right\rfloor\right) &\leq \Pr\left(|Y_1 - E(Y_1)| \geq n^{\frac{1}{2}}\right) \\ &\leq \frac{\text{Var}(Y_1)}{n} \\ &\leq \frac{1}{4} \cdot \frac{\lceil n^{\frac{3}{4}} \rceil}{n} \\ &\leq \frac{1}{4} \cdot \frac{n^{\frac{3}{4}} + 1}{n} \\ &\leq \frac{1}{4} \cdot \frac{2 \cdot n^{\frac{3}{4}}}{n} \\ &= \frac{1}{2} \cdot n^{-\frac{1}{4}} \\ &\leq n^{-\frac{1}{4}}\end{aligned}$$



Utilizando nuevamente la desigualdad de Chebyshev

Lema

$$\Pr\left(Y_2 \leq \lceil n^{\frac{3}{4}} \rceil - \left\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \right\rceil\right) \leq n^{-\frac{1}{4}}$$

Ejercicio

Demuestre el lema utilizando las ideas en la demostración del lema anterior.

Un último uso de la desigualdad de Chebyshev

Lema

$$Pr(\mathbf{Length}(S) > 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor) \leq 4 \cdot n^{-\frac{1}{4}}$$

Demostración. Si $\mathbf{Length}(S) > 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor$, entonces al menos una de las siguientes condiciones debe ser cierta:

- (a) $|\{i \in \{1, \dots, \mathbf{Length}(S)\} \mid S[i] > m\}| \geq 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor$
- (b) $|\{i \in \{1, \dots, \mathbf{Length}(S)\} \mid S[i] < m\}| \geq 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor$

Un último uso de la desigualdad de Chebyshev

Vamos a demostrar que la probabilidad de que (a) ocurra es menor o igual a $2 \cdot n^{-\frac{1}{4}}$

De la misma forma se demuestra que la probabilidad que (b) ocurra es menor o igual a $2 \cdot n^{-\frac{1}{4}}$

► De esto se concluye que $\Pr(\mathbf{Length}(S) > 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor) \leq 4 \cdot n^{-\frac{1}{4}}$

Un último uso de la desigualdad de Chebyshev

Suponga que (a) es cierto, y sea ℓ la posición de u en la lista L ordenada.

- ▶ Tenemos que $\ell \geq \lceil \frac{n}{2} \rceil + 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor$

Dado que $u = R[\lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil]$, al menos $\lceil n^{\frac{3}{4}} \rceil - \lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil$ elementos de R deben estar en posiciones mayores o iguales a ℓ en la lista L ordenada.

- ▶ No podemos asegurar que estos elementos están en posiciones mayores a ℓ puesto que R puede tener elementos repetidos
- ▶ Vamos a acotar superiormente la probabilidad de que esto ocurra para obtener un cota superior para la probabilidad de que (a) ocurra

Un último uso de la desigualdad de Chebyshev

Para cada $i \in \{1, \dots, \lceil n^{\frac{3}{4}} \rceil\}$, definimos una variable aleatoria W_i de la siguiente forma:

$$W_i = \begin{cases} 1 & \text{si la posición de } R[i] \text{ en la lista } L \text{ ordenada} \\ & \text{es mayor o igual a } \lceil \frac{n}{2} \rceil + 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor \\ 0 & \text{en otro caso} \end{cases}$$

Además, definimos la variable aleatoria W como $\sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} W_i$

Dado que $\ell \geq \lceil \frac{n}{2} \rceil + 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor$, tenemos entonces que la probabilidad de que (a) ocurra es menor o igual a:

$$\Pr(W \geq \lceil n^{\frac{3}{4}} \rceil - \lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil)$$

Un último uso de la desigualdad de Chebyshev

Como L no contiene elementos repetidos obtenemos:

$$\begin{aligned}\Pr(W_i = 1) &= \frac{n - (\lceil \frac{n}{2} \rceil + 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor) + 1}{n} \\ &= \frac{\lfloor \frac{n}{2} \rfloor - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor + 1}{n} \\ &= \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n}\end{aligned}$$

Tenemos entonces que:

$$\begin{aligned}E(W_i) &= \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \\ \text{Var}(W_i) &= \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \cdot \left(1 - \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n}\right)\end{aligned}$$

Un último uso de la desigualdad de Chebyshev

Por lo tanto:

$$\begin{aligned} E(W) &= E\left(\sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} W_i\right) \\ &= \sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} E(W_i) \\ &= \sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \\ &= \lceil n^{\frac{3}{4}} \rceil \cdot \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \end{aligned}$$

Un último uso de la desigualdad de Chebyshev

Pero tenemos que:

$$\begin{aligned}\lceil n^{\frac{3}{4}} \rceil \cdot \frac{\lceil \frac{n}{2} \rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} &\leq (n^{\frac{3}{4}} + 1) \cdot \frac{\frac{n}{2} - 2 \cdot n^{\frac{3}{4}} + 3}{n} \\&= n^{\frac{3}{4}} \cdot \frac{\frac{n}{2} - 2 \cdot n^{\frac{3}{4}} + 3}{n} + \frac{\frac{n}{2} - 2 \cdot n^{\frac{3}{4}} + 3}{n} \\&= n^{\frac{3}{4}} \cdot \frac{\frac{n}{2} - 2 \cdot n^{\frac{3}{4}}}{n} + 3 \cdot n^{-\frac{1}{4}} + \frac{1}{2} - 2 \cdot n^{-\frac{1}{4}} + \frac{3}{n} \\&= n^{\frac{3}{4}} \cdot \frac{\frac{n}{2} - 2 \cdot n^{\frac{3}{4}}}{n} + n^{-\frac{1}{4}} + \frac{1}{2} + \frac{3}{n} \\&= \frac{1}{2} \cdot n^{\frac{3}{4}} - 2 \cdot n^{\frac{1}{2}} + n^{-\frac{1}{4}} + \frac{1}{2} + \frac{3}{n} \\&\leq \frac{1}{2} \cdot n^{\frac{3}{4}} - 2 \cdot n^{\frac{1}{2}} + 1\end{aligned}$$

Concluimos que $E(W) \leq \frac{1}{2} \cdot n^{\frac{3}{4}} - 2 \cdot n^{\frac{1}{2}} + 1$

Un último uso de la desigualdad de Chebyshev

Para $i, j \in \{1, \dots, \lceil n^{\frac{3}{4}} \rceil\}$ tal que $i \neq j$ se tiene que W_i es independiente de W_j

Concluimos entonces que:

$$\begin{aligned}\text{Var}(W) &= \text{Var}\left(\sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} W_i\right) \\&= \sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} \text{Var}(W_i) \\&= \sum_{i=1}^{\lceil n^{\frac{3}{4}} \rceil} \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \cdot \left(1 - \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n}\right) \\&= \lceil n^{\frac{3}{4}} \rceil \cdot \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n} \cdot \left(1 - \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n}\right) \\&\leq \lceil n^{\frac{3}{4}} \rceil \cdot \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \lfloor n^{\frac{3}{4}} \rfloor}{n}\end{aligned}$$

Un último uso de la desigualdad de Chebyshev

Además tenemos que:

$$\begin{aligned}\left\lceil n^{\frac{3}{4}} \right\rceil \cdot \frac{\left\lceil \frac{n}{2} \right\rceil - 2 \cdot \left\lfloor n^{\frac{3}{4}} \right\rfloor}{n} &\leq \frac{1}{2} \cdot n^{\frac{3}{4}} - 2 \cdot n^{\frac{1}{2}} + 1 \\ &\leq \frac{1}{2} \cdot n^{\frac{3}{4}} + 1 \\ &\leq n^{\frac{3}{4}}\end{aligned}$$

Concluimos que $\text{Var}(W) \leq n^{\frac{3}{4}}$

Un último uso de la desigualdad de Chebyshev

Finalmente tenemos que:

$$\begin{aligned}\Pr(W \geq \lceil n^{\frac{3}{4}} \rceil - \lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil) &\leq \Pr(W \geq n^{\frac{3}{4}} - \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} - 1) \\&= \Pr(W \geq \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} - 1) \\&= \Pr(W \geq \frac{1}{2} \cdot n^{\frac{3}{4}} - 2 \cdot n^{\frac{1}{2}} + 1 + n^{\frac{1}{2}} - 2) \\&\leq \Pr(W \geq E(W) + n^{\frac{1}{2}} - 2) \\&\leq \Pr(W \geq E(W) + n^{\frac{1}{2}} - (1 - \frac{1}{\sqrt{2}}) \cdot n^{\frac{1}{2}}) \\&\leq \Pr(W \geq E(W) + \sqrt{\frac{n}{2}}) \\&= \Pr(W - E(W) \geq \sqrt{\frac{n}{2}}) \\&\leq \Pr(|W - E(W)| \geq \sqrt{\frac{n}{2}})\end{aligned}$$

Un último uso de la desigualdad de Chebyshev

Por lo tanto, utilizando la desigualdad de Chebyshev concluimos que:

$$\begin{aligned}\Pr(W \geq \lceil n^{\frac{3}{4}} \rceil - \lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil) &\leq \Pr(|W - E(W)| \geq \sqrt{\frac{n}{2}}) \\ &\leq \frac{\text{Var}(W)}{\frac{n}{2}} \\ &\leq \frac{n^{\frac{3}{4}}}{\frac{n}{2}} \\ &= 2 \cdot n^{-\frac{1}{4}}\end{aligned}$$

Concluimos que la probabilidad de que (a) ocurra es menor o igual a $2 \cdot n^{-\frac{1}{4}}$.



El cálculo final

Recuerde que estamos considerando una lista $L[1 \dots n]$ de números enteros donde n es impar y mayor o igual a 2001.

Para la lista L demostramos lo siguiente:

$$\begin{aligned}\Pr(Y_1 < \lfloor \frac{1}{2} \cdot n^{\frac{3}{4}} - n^{\frac{1}{2}} \rfloor) &\leq n^{-\frac{1}{4}} \\ \Pr(Y_2 \leq \lceil n^{\frac{3}{4}} \rceil - \lceil \frac{1}{2} \cdot n^{\frac{3}{4}} + n^{\frac{1}{2}} \rceil) &\leq n^{-\frac{1}{4}} \\ \Pr(\mathbf{Length}(S) > 4 \cdot \lfloor n^{\frac{3}{4}} \rfloor) &\leq 4 \cdot n^{-\frac{1}{4}}\end{aligned}$$

El cálculo final

Tenemos entonces que:

$$\Pr(\text{CalcularMediana}(L) \text{ retorne } \textit{sin_resultado}) \leq 6 \cdot n^{-\frac{1}{4}}$$

Dado que $n \geq 2001$ concluimos que

$$\begin{aligned} \Pr(\text{CalcularMediana}(L) \text{ retorne } \textit{sin_resultado}) &\leq 6 \cdot 2001^{-\frac{1}{4}} \\ &< \frac{9}{10} \end{aligned}$$

El tiempo esperado del algoritmo

Sea p la probabilidad de que **CalcularMediana**(L) retorne resultado

▶ Tenemos que $p \geq \frac{1}{10}$

¿En promedio cuántas veces se debe llamar a **CalcularMediana**(L) para obtener la mediana de la lista L ?

El tiempo esperado del algoritmo

Sea T una variable aleatoria tal que para cada $i \geq 1$:

$$\Pr(T = i) = (1 - p)^{i-1} \cdot p$$

Vale decir, T representa el número de llamadas a **CalcularMediana**(L) hasta obtener un resultado

Dado que T tiene distribución geométrica de parámetro p , concluimos que

$$E(T) = \frac{1}{p} \leq 10$$

Concluimos que en promedio se debe llamar 10 veces a **CalcularMediana**(L) para obtener la mediana de la lista L